



DATOS DEL ESTUDIANTE

Apellidos y Nombres:	VELASQUEZ ALVA KARLA LUCERO ARIANA	ID:	1565857
Dirección Zonal/CFP:	VIRTUAL		
Carrera:	INGENIERIA DE SOFTWARE CON IA	Semestre:	IV
Curso/ Mód. Formativo:			
Tema de Trabajo Final:	Inteligencia Artificial, Machine Learning, Deep Learning y Manipulación Numérica en Python		

Proyecto Integrador: Inteligencia Artificial, Machine Learning, Deep Learning y Manipulación Numérica en Python**1. Introducción**

El siguiente proyecto tiene como objetivo integrar conocimientos teóricos y prácticos en Inteligencia Artificial (IA), Machine Learning (ML) y Deep Learning (DL), mientras se demuestra el dominio de operaciones numéricas y manipulación de matrices utilizando Python. Se diseñará y desarrollará un modelo predictivo basado en redes neuronales profundas para la clasificación de imágenes, enfatizando el procesamiento de datos numéricos y la optimización computacional.

2. Descripción General del Proyecto

Título: Clasificación de Imágenes de Dígitos Manuscritos Usando Redes Neuronales Profundas

Objetivo: Desarrollar un modelo de Deep Learning que clasifique dígitos manuscritos (0-9) del conjunto de datos MNIST, haciendo especial énfasis en la manipulación explícita de matrices y operaciones numéricas.

Herramientas:

- numpy para operaciones matriciales
- pandas para manipulación de datos
- matplotlib para visualización
- tensorflow o pytorch para la construcción de redes neuronales
- scikit-learn para evaluación de modelos

3. Marco Teórico

3.1 Inteligencia Artificial (IA): Disciplina que busca crear sistemas capaces de realizar tareas que normalmente requieren inteligencia humana.

3.2 Machine Learning (ML): Subcampo de la IA que estudia algoritmos que mejoran su rendimiento a través de la experiencia.

3.3 Deep Learning (DL): Subcampo del ML que utiliza arquitecturas de redes neuronales profundas para aprender representaciones jerárquicas de los datos.

3.4 Operaciones Numéricas y Manipulación de Matrices: Conceptos esenciales para el manejo de grandes cantidades de datos y la implementación eficiente de algoritmos de aprendizaje automático.

4. Desarrollo del Proyecto

4.1 Procesamiento de Datos

- **Carga de Datos:**

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

- **Normalización:**

```
x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255
```

- **Reformateo:**

```
x_train = x_train.reshape((-1, 28*28))
x_test = x_test.reshape((-1, 28*28))
```

- **Visualización de una Imagen:**

```
import matplotlib.pyplot as plt
plt.imshow(x_train[0].reshape(28,28), cmap='gray')
plt.title(f"Etiqueta: {y_train[0]}")
plt.show()
```

4.2 Manipulación Numérica

- **Cálculo manual de la media y varianza de los datos:**

```
import numpy as np
media = np.mean(x_train, axis=0)
varianza = np.var(x_train, axis=0)
```

- **Operaciones matriciales:** Cálculo del producto de matrices para la inicialización de pesos.

```
pesos_iniciales = np.random.randn(28*28, 128) * np.sqrt(1. / 128)
bias_iniciales = np.zeros(128)
```

4.3 Construcción del Modelo

- **Red Neuronal Simple:**

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
modelo = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
modelo.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

- **Entrenamiento del Modelo:**

```
modelo.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
```

- **Evaluación:**

```
test_loss, test_accuracy = modelo.evaluate(x_test, y_test)
print(f"Precisión en test: {test_accuracy:.4f}")
```

4.4 Análisis de Resultados

- **Matriz de Confusión:**

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
predicciones = np.argmax(modelo.predict(x_test), axis=1)
matriz_confusion = confusion_matrix(y_test, predicciones)
```

```
sns.heatmap(matriz_confusion, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicción')
plt.ylabel('Etiqueta Verdadera')
plt.show()
```

- **Comentarios:**

- Analizar en qué dígitos ocurren más errores.
- Proponer mejoras (mayor profundidad, regularización, etc.).

5. Conclusiones

Este proyecto muestra la integración efectiva de conceptos de IA, ML y DL con una fuerte base numérica. El procesamiento de datos y la manipulación de matrices han sido fundamentales en la preparación y entrenamiento del modelo. A partir de los resultados, es posible proponer mejoras futuras como el ajuste de hiperparámetros o el uso de arquitecturas más complejas (CNNs).

Respuestas a preguntas guía

Durante el análisis y estudio del caso práctico, debes obtener las respuestas a las interrogantes:

Pregunta 01:	Diferencia entre IA, Machine Learning, y Deep Learning
La Inteligencia Artificial (IA) es un campo amplio que busca crear sistemas capaces de realizar tareas que requieren inteligencia humana, como razonar, aprender o resolver problemas. Dentro de la IA, el Machine Learning (ML) es un subcampo que se enfoca en desarrollar algoritmos que permiten a las máquinas aprender de datos y mejorar su desempeño sin ser programadas explícitamente. A su vez, el Deep Learning (DL) es una rama del ML que utiliza redes neuronales profundas, inspiradas en el cerebro humano, para procesar grandes volúmenes de datos y extraer patrones complejos. Mientras la IA es el objetivo general, el ML es una técnica específica para alcanzar ese objetivo, y el DL es una técnica aún más especializada dentro del ML. En IA pueden incluirse métodos que no aprenden de datos, como los sistemas basados en reglas. El ML, en cambio, siempre implica aprendizaje a partir de ejemplos o experiencias. El DL es especialmente potente para tareas como visión por computadora, procesamiento de lenguaje natural y reconocimiento de voz.	
Pregunta 02:	Cuál es la diferencia entre aprendizaje supervisado, no supervisado y por reforzamiento? Da un ejemplo de cada uno.
El aprendizaje supervisado es un tipo de Machine Learning donde el modelo se entrena con datos de entrada y sus correspondientes salidas correctas (etiquetas), para luego predecir o clasificar nuevos datos; un ejemplo es un sistema que reconoce si un correo es <i>spam</i> o <i>no spam</i> . El aprendizaje no supervisado trabaja con datos que no tienen etiquetas, buscando patrones o estructuras ocultas de manera automática; un ejemplo es un algoritmo que segmenta clientes en grupos de compra similares. Por último, el aprendizaje por reforzamiento implica que un agente aprende a actuar en un entorno tomando decisiones y recibiendo recompensas o penalizaciones según el resultado; un ejemplo es un robot que aprende a caminar optimizando sus movimientos para no caer.	

Pregunta 03:	¿Qué papel juegan los conjuntos de datos (datasets) en el rendimiento de un modelo de ML?
Los conjuntos de datos (datasets) son fundamentales para el rendimiento de un modelo de Machine Learning porque proporcionan la "experiencia" de la cual el modelo aprende. Un dataset de alta calidad —grande, diverso, bien etiquetado y representativo del problema— permite que el modelo generalice mejor y obtenga un rendimiento superior en datos nuevos. Si el dataset es pequeño, tiene errores, sesgos o no cubre bien los casos posibles, el modelo puede sobreajustarse (memorizar los datos en lugar de aprender patrones) o funcionar mal en situaciones reales. Además, la cantidad, variedad y equilibrio de los datos afectan directamente la capacidad del modelo para ser preciso, justo y robusto.	
Pregunta 04:	¿Cuáles son los principales algoritmos usados en aprendizaje supervisado, y en qué casos se suele escoger cada uno?
En aprendizaje supervisado, los principales algoritmos son: • Regresión Lineal: Se usa para predecir valores continuos, como precios de casas o temperaturas, cuando la relación entre variables parece ser lineal. • Regresión Logística: Ideal para problemas de clasificación binaria, como diagnosticar si un paciente tiene una enfermedad o no. • Máquinas de Vectores de Soporte (SVM): Se eligen para problemas de clasificación donde se busca maximizar el margen entre clases, especialmente útiles en conjuntos pequeños y de alta dimensión. • Árboles de Decisión: Son intuitivos y fáciles de interpretar, usados tanto para clasificación como regresión, especialmente cuando se necesita explicar fácilmente el modelo. • Random Forest: Un ensamble de muchos árboles de decisión, se usa cuando se quiere mejorar la precisión y reducir el sobreajuste. • k-Nearest Neighbors (k-NN): Se aplica en problemas donde la similitud entre ejemplos es clave, aunque puede ser ineficiente con grandes volúmenes de datos. • Redes Neuronales: Se eligen para problemas complejos y no lineales, como el reconocimiento de imágenes o voz, aunque requieren muchos datos y poder de cómputo.	

1. PLANIFICACIÓN DEL TRABAJO

▪ Cronograma de actividades:

N°	ACTIVIDADES	CRONOGRAMA					
		25/04	28/04	29/04	30/04		
01	Elaboracion del Informe (Formato del alumno)	X					
02	Elaboracion del codigo		X				
03	Elaboracion del PPT			X			
04	Subir el informe (formato PDF) y el script (enlace de Github)				X		
05							
06							

▪ Lista de recursos necesarios:

1. MÁQUINAS Y EQUIPOS	
Descripción	Cantidad
Laptop	1
Celular	1

2. HERRAMIENTAS E INSTRUMENTOS	
Descripción	Cantidad
Editor de código (visual estudio code)	1

3. MATERIALES E INSUMOS	
Descripción	Cantidad
Documento de referencia	1

2. DECIDIR PROPUESTA

- Describe la propuesta determinada para la solución del caso práctico

PROPUESTA DE SOLUCIÓN

Para resolver el problema de clasificación de dígitos manuscritos, se propone construir un modelo de red neuronal densa (Fully Connected Neural Network) que trabaje con las imágenes aplanadas (matrices de 28x28 transformadas en vectores de 784 componentes). Se aplicará normalización de datos para mejorar el rendimiento y acelerar la convergencia durante el entrenamiento. La manipulación de matrices y operaciones numéricas serán una parte integral de la preparación de datos, inicialización de pesos y análisis de resultados.

El modelo consistirá en varias capas densas con funciones de activación ReLU, y finalizará con una capa de salida de 10 neuronas usando softmax para predicciones multiclase. El entrenamiento se realizará usando el optimizador Adam y la función de pérdida Sparse Categorical Crossentropy. La evaluación del modelo incluirá el análisis de la matriz de confusión para identificar patrones de error y posibles mejoras.

3. EJECUTAR

- Resolver el caso práctico, utilizando como referencia el problema propuesto y las preguntas guía proporcionadas para orientar el desarrollo.
- Fundamentar sus propuestas en los conocimientos adquiridos a lo largo del curso, aplicando lo aprendido en las tareas y operaciones descritas en los contenidos curriculares.

INSTRUCCIONES: Ser lo más explícito posible. Los gráficos ayudan a transmitir mejor las ideas. Tomar en cuenta los aspectos de calidad, medio ambiente y SHI.

OPERACIONES / PASOS / SUBPASOS
Análisis del Problema Propuesto Caso práctico: Clasificación automática de imágenes de dígitos manuscritos (0-9) usando modelos de <i>Deep Learning</i>, con énfasis en operaciones numéricas, manipulación de matrices y uso de Python como herramienta principal.
Desarrollo del Caso con Fundamento Técnico <i>1. Carga y Preprocesamiento de los Datos</i> Se utiliza el dataset MNIST, el cual contiene 70,000 imágenes (60,000 para entrenamiento y 10,000 para prueba), en escala de grises, de tamaño 28x28 píxeles. <pre> from tensorflow.keras.datasets import mnist (x_train, y_train), (x_test, y_test) = mnist.load_data() # Normalización x_train = x_train.astype('float32') / 255 x_test = x_test.astype('float32') / 255 # Aplanamiento x_train = x_train.reshape((-1, 784)) x_test = x_test.reshape((-1, 784)) </pre> • <i>Fundamento:</i> El preprocesamiento es crucial para mejorar la eficiencia del entrenamiento. La normalización garantiza que los datos estén en un rango apropiado [0,1].
<i>2. Visualización de Datos</i> <pre> import matplotlib.pyplot as plt plt.imshow(x_train[0].reshape(28,28), cmap='gray') plt.title(f'Etiqueta: {y_train[0]}') plt.axis("off") plt.show() </pre> - <i>Fundamento:</i> Visualizar los datos es esencial para validar que el preprocesamiento se ha hecho correctamente y para entender mejor el problema.

3. Análisis Numérico y Manipulación de Matrices

Cálculo de estadísticas básicas:

```
import numpy as np
```

```
media = np.mean(x_train, axis=0)
```

```
varianza = np.var(x_train, axis=0)
```

Inicialización de pesos con operaciones matriciales:

```
pesos_iniciales = np.random.randn(784, 128) * np.sqrt(1. / 128)
```

```
bias_iniciales = np.zeros(128)
```

- *Fundamento*: Estas operaciones muestran comprensión del manejo de datos numéricos. La inicialización adecuada de pesos evita el problema de *vanishing gradients*.

4. Construcción y Entrenamiento del Modelo

Modelo secuencial simple:

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
modelo = Sequential([  
    Dense(128, activation='relu', input_shape=(784,)),  
    Dense(64, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

```
modelo.compile(optimizer='adam',          loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

```
modelo.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
```

- *Fundamento*: Este modelo emplea capas densas (*fully connected*) y funciones de activación ReLU y softmax, siendo ideales para clasificación multiclase. El optimizador adam mejora la convergencia.

5. Evaluación y Análisis de Resultados

```
test_loss, test_accuracy = modelo.evaluate(x_test, y_test)
```

```
print(f"Precisión en test: {test_accuracy:.4f}")
```

```
python
```

```
CopiarEditar
```

```
from sklearn.metrics import confusion_matrix
```

```
import seaborn as sns
```

```
predicciones = np.argmax(modelo.predict(x_test), axis=1)
```

```
matriz_confusion = confusion_matrix(y_test, predicciones)
```

```
sns.heatmap(matriz_confusion, annot=True, fmt='d', cmap='Blues')
```

```
plt.xlabel('Predicción')
```

```
plt.ylabel('Etiqueta Verdadera')
```

```
plt.title('Matriz de Confusión')
```

```
plt.show()
```

Análisis de errores:

- Errores comunes entre clases visualmente similares (por ejemplo, 4 y 9, 5 y 6).
- La matriz de confusión permite identificar patrones de equivocación sistemáticos.

- *Fundamento:* Las métricas y visualizaciones como la matriz de confusión son esenciales para evaluar el rendimiento más allá de la precisión.

DIBUJO / ESQUEMA / DIAGRAMA DE PROPUESTA

(Adicionar las páginas que sean necesarias)

```
import numpy as np

def validar_matriz(matriz, nombre='matriz'):
    """Valida que la entrada sea una matriz numpy válida."""
    if matriz is None:
        raise ValueError(f'{nombre} no puede ser None.')
    if not isinstance(matriz, np.ndarray):
        raise TypeError(f'{nombre} debe ser un objeto numpy.ndarray.')
    if matriz.size == 0:
        raise ValueError(f'{nombre} no puede estar vacía.')
    if np.isnan(matriz).any():
        raise ValueError(f'{nombre} contiene valores nulos (NaN).')

def generar_matriz(filas, columnas, minimo=0, maximo=10, tipo=int):
    """Genera una matriz aleatoria."""
    if filas <= 0 or columnas <= 0:
        raise ValueError("Las dimensiones deben ser positivas.")
    if tipo == int:
        matriz = np.random.randint(minimo, maximo, size=(filas, columnas))
    elif tipo == float:
        matriz = np.random.uniform(minimo, maximo, size=(filas, columnas))
    else:
        raise TypeError("Tipo no soportado. Usa int o float.")
    return matriz

def propiedad_conmutativa(A, B):
    """Verifica la propiedad conmutativa: A + B = B + A"""
    validar_matriz(A, "A")
    validar_matriz(B, "B")
    if A.shape != B.shape:
        raise ValueError("Las matrices deben tener las mismas dimensiones para la suma.")
    return np.allclose(A + B, B + A)

def propiedad_asociativa(A, B, C):
    """Verifica la propiedad asociativa: (A + B) + C = A + (B + C)"""
    validar_matriz(A, "A")
    validar_matriz(B, "B")
    validar_matriz(C, "C")
    if A.shape != B.shape or B.shape != C.shape:
        raise ValueError("Todas las matrices deben tener las mismas dimensiones para la suma.")
    return np.allclose((A + B) + C, A + (B + C))

def propiedad_distributiva(A, B, C):
    """Verifica la propiedad distributiva: A*(B + C) = A*B + A*C"""
    validar_matriz(A, "A")
    validar_matriz(B, "B")
```

```

validar_matriz(C, "C")
if B.shape != C.shape:
    raise ValueError("B y C deben tener las mismas dimensiones para la suma.")
if A.shape[1] != B.shape[0]:
    raise ValueError("El número de columnas de A debe coincidir con el número de
filas de B y C para la multiplicación.")
izquierda = A @ (B + C)
derecha = (A @ B) + (A @ C)
return np.allclose(izquierda, derecha)

def propiedad_inverso(A):
    """Verifica el inverso aditivo:  $A + (-A) = 0$ """
    validar_matriz(A, "A")
    inverso = -A
    suma = A + inverso
    return np.allclose(suma, np.zeros_like(A))

def propiedad_identidad(A):
    """Verifica la identidad aditiva:  $A + 0 = A$ """
    validar_matriz(A, "A")
    identidad = np.zeros_like(A)
    return np.allclose(A + identidad, A)

# Ejemplo de uso:
if __name__ == "__main__":
    A = generar_matriz(3, 3)
    B = generar_matriz(3, 3)
    C = generar_matriz(3, 3)

    print("Matriz A:\n", A)
    print("Matriz B:\n", B)
    print("Matriz C:\n", C)

    print("\nPropiedad conmutativa:", propiedad_conmutativa(A, B))
    print("Propiedad asociativa:", propiedad_asociativa(A, B, C))
    print("Propiedad distributiva:", propiedad_distributiva(A, B, C))
    print("Propiedad inverso aditivo:", propiedad_inverso(A))
    print("Propiedad identidad aditiva:", propiedad_identidad(A))

```

 SENATI	[NOMBRE DEL TEMA DEL TRABAJO FINAL]	
	[APELLIDOS Y NOMBRES]	[ESCALA]

4. CONTROLAR

- Verificar el cumplimiento de los procesos desarrollados en la propuesta de solución del caso práctico.

EVIDENCIAS	CUMPLE	NO CUMPLE
• ¿Se identificó claramente la problemática del caso práctico?	☒	☐
• ¿Se desarrolló las condiciones de los requerimientos solicitados?	☒	☐
• ¿Se formularon respuestas claras y fundamentadas a todas las preguntas guía?	☒	☐
• ¿Se elaboró un cronograma claro de actividades a ejecutar?	☒	☐
• ¿Se identificaron y listaron los recursos (máquinas, equipos, herramientas, materiales) necesarios para ejecutar la propuesta?	☒	☐
• ¿Se ejecutó la propuesta de acuerdo con la planificación y cronograma establecidos?	☒	☐
• ¿Se describieron todas las operaciones y pasos seguidos para garantizar la correcta ejecución?	☒	☐
• ¿La propuesta es pertinente con los requerimientos solicitados?	☒	☐
• ¿Se evaluó la viabilidad de la propuesta para un contexto real?	☒	☐

5. VALORAR

- Califica el impacto que representa la propuesta de solución ante la situación planteada en el caso práctico.

CRITERIO DE EVALUACIÓN	DESCRIPCIÓN DEL CRITERIO	PUNTUACIÓN MÁXIMA	PUNTAJE CALIFICADO POR EL ESTUDIANTE
Identificación del problema	Claridad en la identificación del problema planteado.	3	
Relevancia de la propuesta de solución	La propuesta responde adecuadamente al problema planteado y es relevante para el contexto del caso práctico.	8	
Viabilidad técnica	La solución es técnicamente factible, tomando en cuenta los recursos y conocimientos disponibles.	6	
Cumplimiento de Normas	La solución cumple con todas las normas técnicas de seguridad, higiene y medio ambiente.	3	
PUNTAJE TOTAL		20	

